

AI Website Summariser

Technical architecture, implementation decisions, and setup guide

Stack: React + Vite frontend · Node.js + Express API · Axios + Cheerio extraction · Groq LLM summarisation

Overview

This full-stack application accepts a public webpage URL and returns a concise factual summary. It keeps browser-facing UI concerns separate from server-side web fetching and AI-provider communication, so API keys remain private and the provider can be replaced without changing the frontend.

Architecture

React/Vite client → Express `POST /api/summarize` → Axios/Cheerio scraper → Groq Chat Completions API

- **Client:** React components provide URL entry, loading feedback, error presentation, and a summary card. A small API module centralises calls to the backend.
- **API:** Express exposes a health route and the summarisation route. CORS permits the Vite development server to access the local API.
- **Scraping service:** Axios downloads public HTML; Cheerio removes non-content elements such as scripts, styles, navigation, and footers, then extracts readable headings, paragraphs, lists, and table cells.
- **AI service:** The cleaned, bounded page text and title are sent to Groq's OpenAI-compatible chat-completions endpoint with a prompt requesting a neutral 4–6 sentence summary.

Key decisions

Vite and React

Vite provides a fast, minimal local development experience and a straightforward production build. React makes the UI state—idle, loading, successful result, and error—clear and componentised.

Backend proxy instead of browser-side fetching

Public websites often block browser cross-origin requests (CORS), and an AI key must never be sent to the browser. The Express server performs both retrieval and AI calls; the browser only communicates with the application's own API. Development CORS is enabled only to allow the separately hosted Vite client to call that API.

Groq provider integration

The app uses Groq with `llama-3.1-8b-instant` for a fast free-tier-friendly implementation. This was chosen in place of Gemini for this build. Provider-specific code is contained in `server/services/ai.service.js`, so Gemini or OpenAI can be adopted later by replacing that service while leaving routes and UI unchanged.

Resilience and safety

The API validates HTTP(S) URLs before fetching, reports insufficient extractable text, handles unreachable pages and timeouts, limits input text before the model call, and reads the Groq key only from server environment variables. `.env` is excluded from version control.

Local setup

1. Install Node.js 18 or newer and obtain a Groq API key from `console.groq.com/keys`.
2. In `server`, run `npm install`, copy `.env.example` to `.env`, set `GROQ_API_KEY`, then run `npm start`.
3. In another terminal, enter `client`, run `npm install`, then `npm run dev`.
4. Open the Vite URL shown in the terminal (normally `http://localhost:5173`) and submit a public article URL.

Verification

The frontend production build is executed with `npm run build`. The backend can be checked at `GET /api/health`; URL validation is exercised through `POST /api/summarize` with an invalid URL. A live summary additionally requires a valid `GROQ_API_KEY` and a publicly reachable target page.

Prepared for the Indus Net Technologies assignment submission.